Lecture 5: Assembly Boot

• Professor G. Sandoval

• Some slides adapted by G. Sandoval for CS3224, from Brendan Dolan-Gavitt and Bryant and O'Hallaron, Computer Systems: A programmer's Perspective, Third Edition

When poll is active, respond at **pollev.com/gsandoval**

Text **GSANDOVAL** to **22333** once to join

How would you rate the difficulty of HW1?

Easy

Just Right

Pushed me a little but not too bad

TOO HARD !!!

Powered by Poll Everywhere

Start the presentation to see live content. For screen share software, share the entire screen. Get help at **pollev.com/app**

Today: Machine Programming Basics

- **Assembly**
  - **More examples**
  - **Function calls**

- **Booting the OS**
  - BIOS

From last class

Address	Value	Register	Value
0x100	0xFF	%rax	0x100
0x104	0xAB	%rcx	0x1
0x108	0x13	%rdx	0x3
0x10C	0x11		

Operand	Value	Comment
%rax		
0x104		
$0x108		
(%rax)		
4(%rax)		

Assembly: Jumps

Instruction	Effect
`jmp`	Always jump
`je/jz`	Jump if eq / zero
`jne/jnz`	Jump if !eq / !zero
`jg`	Jump if greater
`jge`	Jump if greater / eq
`jl`	Jump if less
`jle`	Jump if less / eq

Function Calls

- Supports creation of functions
- call <f>: push the address of next instruction on the stack and jump to f

- Example:

```
1000 call <f>          // pushes 5 and jumps to f
1005 movl %eax, %ebx
```

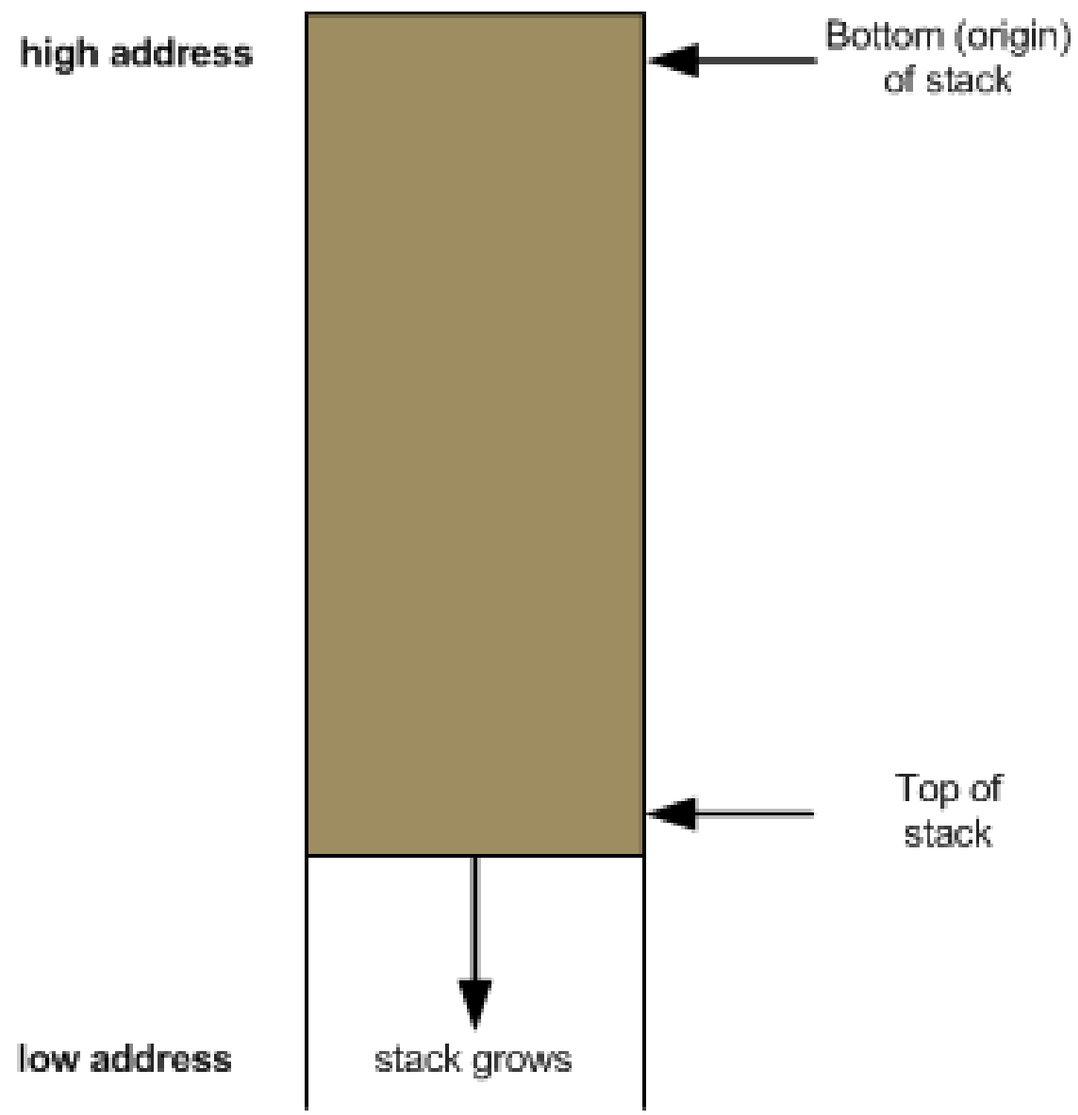
- ret: pop return address off the stack and jump to it

Stack Frames

- **x86 programs use the stack to support function calls.**
- **The machine uses the stack to:**
  - Pass arguments
  - store return information
  - Save registers
  - Local Storage
- **Therefore to use arguments and return information we need to deal with the stack.**
- **The portion of memory used in the stack for a single procedure call is called a stack frame.**

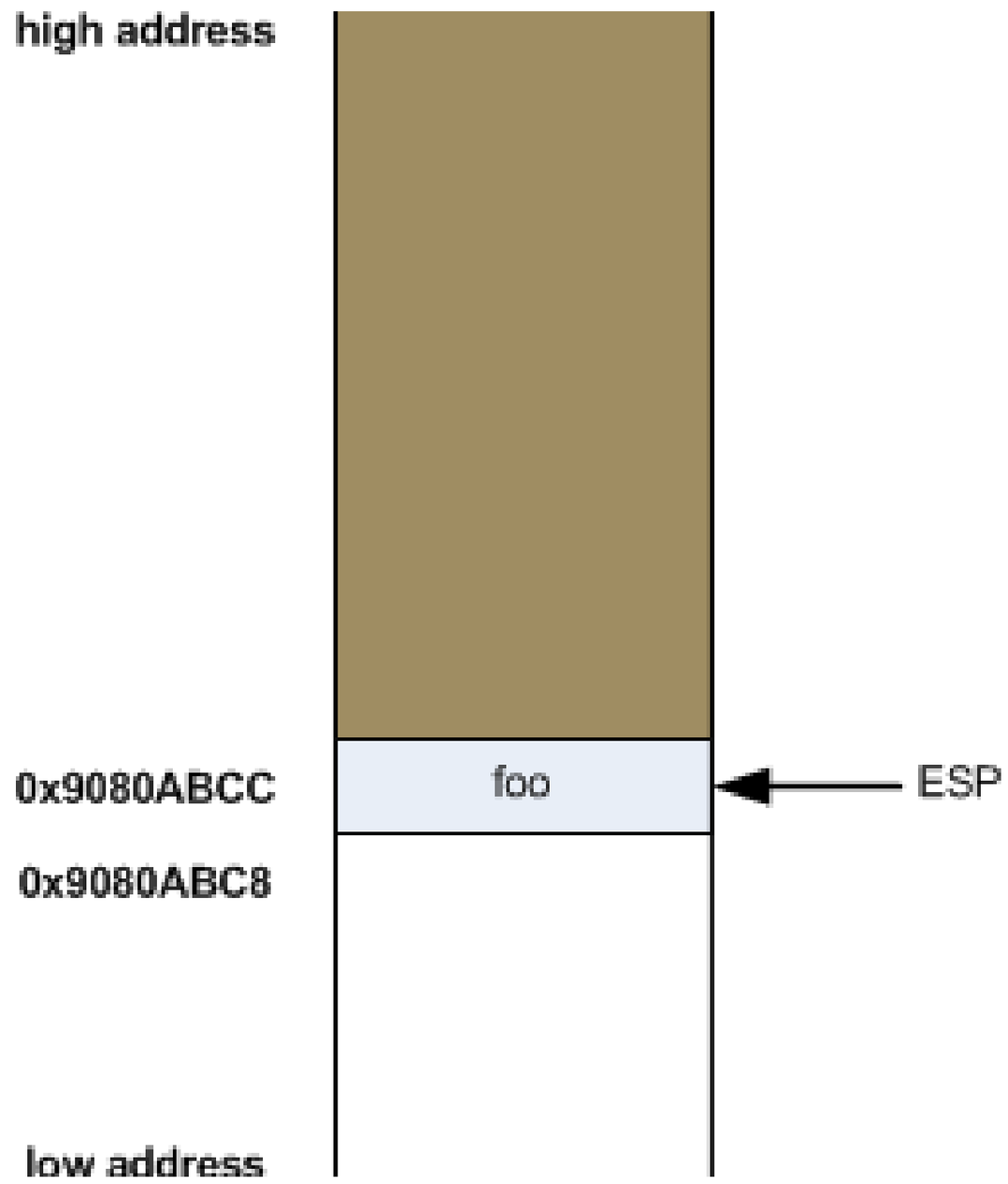
The stack in x86

- The stack in x86 grows down!
- When we say "top of the stack " on x86 we actually mean the lowest address in the memory area occupied by the stack.
- This might be "unnatural" but keep it in mind.



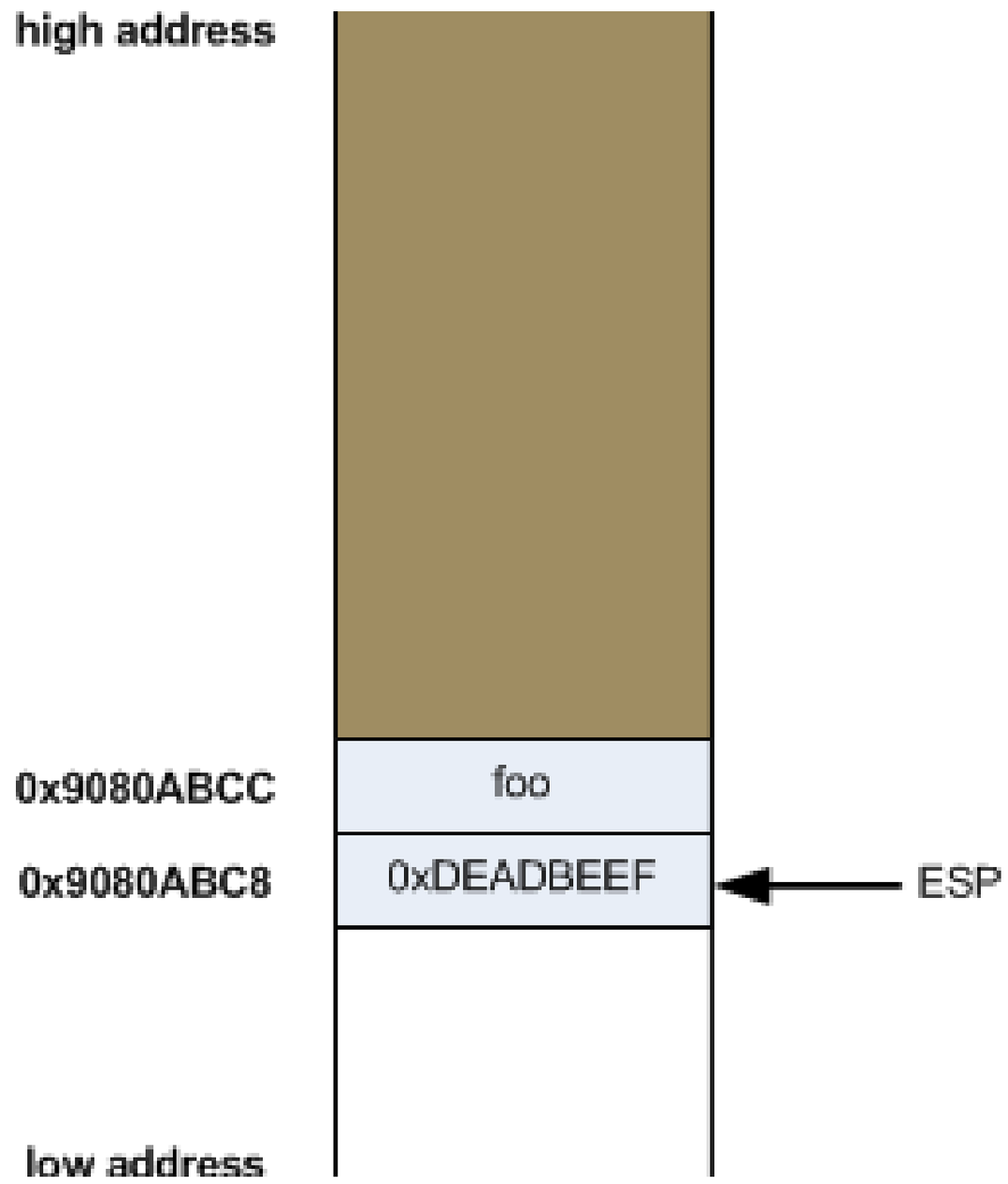
Pushing and Popping data

- We push data with the stack pointer
- `0x9080ABCC` top of stack
- The word at that address has "foo"
- ESP has the address `0x9080ABCC`



Pushing and Popping data

- **To push new data we use the push instruction.**
- **push**
  - decrement esp by 4
  - Store its operand in the location esp points to
- **Example:**
  - Push eax



Stack Frames

- The topmost stack frame is delimited by two register pointers:
  - **%ebp** is the frame pointer (base)
  - **%esp** is the stack pointer
- **So stack arguments can be accessed with %(ebp)**

Stack Usage

```
          +-------------+    |
          | arg 2       |    \
          +-------------+     >- previous function's stack frame
          | arg 1       |    /
          +-------------+    |
          | ret %eip    |    /
          +=============+
          | saved %ebp  |    \
  %ebp-> +-------------+    |
          |             |    |
          |   local     |    \
          | variables,  |     >- current function's stack frame
          |    etc.     |    /
          |             |    |
          |             |    |
  %esp-> +-------------+    /
```

Function Prologue & Epilogue

- **The need to set up a stack frame gives rise to a common function prologue:**

```
push %ebp              // save old value of ebp
mov %esp, %ebp         // set base pointer
sub $X, %esp           // reserve space for locals
```

- **At the end, we can clean up with just the epilogue**

```
mov %ebp, %esp         // restore stack pointer
pop %ebp               // restore base pointer
ret                    // return pops return address
```

Stack Usage

- **GCC dictates how the stack is used. Contract between caller and callee on x86**

- **Entry to a function: (just after call)**
  - %eip points at the first instruction of the function
  - %ebp points to the return address
  - %ebp+4 points at first argument to function

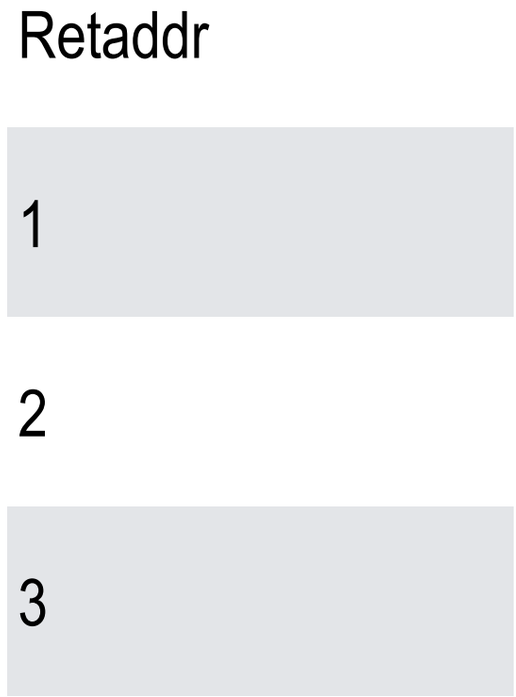
- **After ret instruction**
  - %eip contains return address
  - %esp points at argument pushed by caller
  - Called function may have trashed arguments
  - %eax contains return value (or thrash if function is void)
  - %eax, %ecx and %edx may be trashed

Calling Conventions

- **A *calling convention* is a system-defined convention that tells you how to call a function**
- **In general a calling convention will:**
  - Say how to pass arguments to the function
  - Say who is responsible for cleaning up arguments
  - Specify where the return value will be placed
  - Say which registers will be preserved by the callee and which might be trashed

gcc Calling Convention in x86 (32-bit)

- **Return value: %eax**
- **Arguments are passed on the stack:**
  - f(1, 2, 3)
  - push 3
    push 2
    push 1
    call <f>



gcc Calling Convention (32-bit)

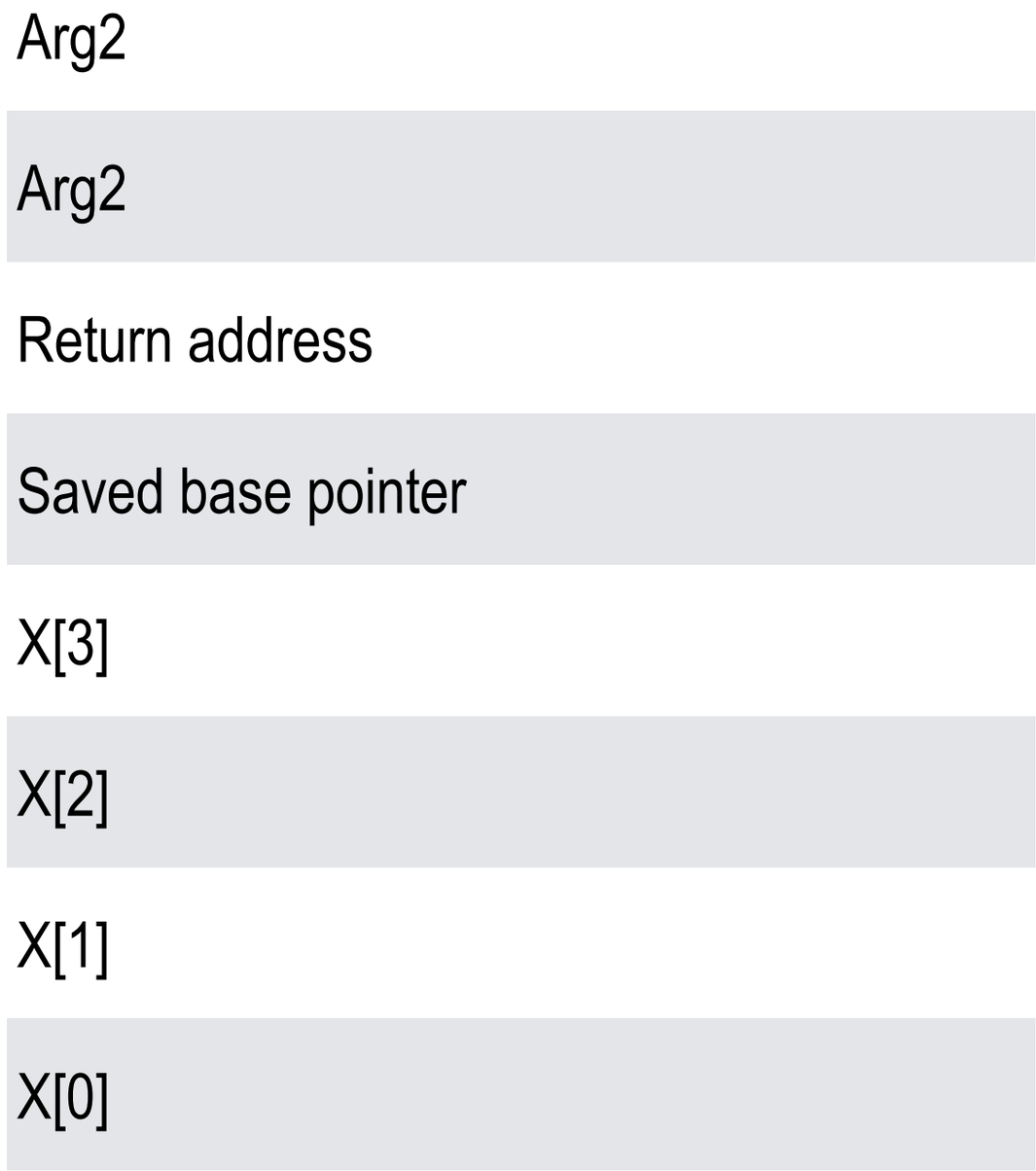
- **Caller is responsible for cleaning arguments off the stack**
  - Note that this allows functions with variable numbers of arguments (varargs)
- **%eax %ecx, %edx may be trashed by callee (must be saved by caller)**
- **%ebp, %ebx, %esi, %edi must be preserved by callee**

Aside: Buffer Overflow

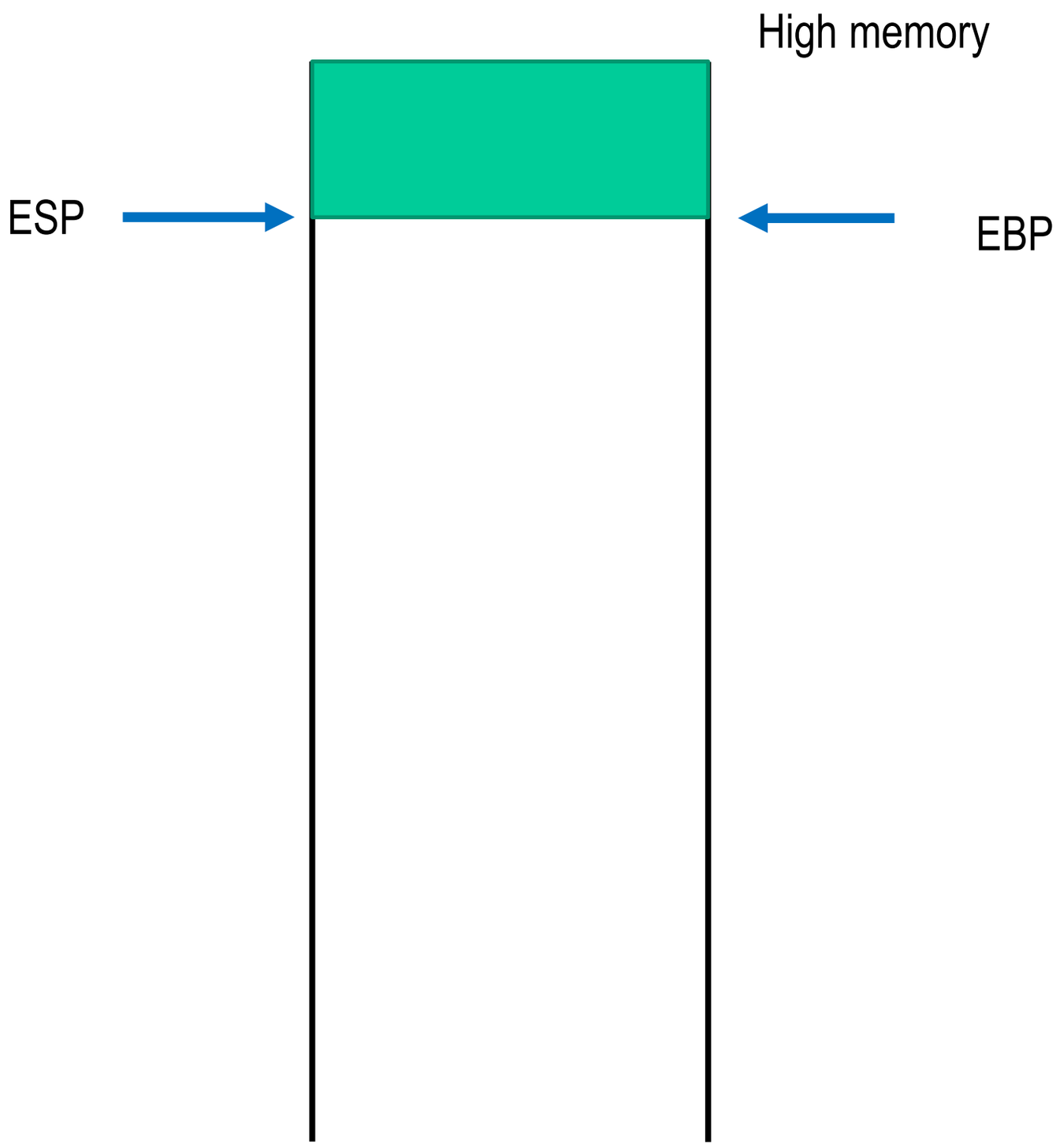
- **Consider a function like:**

```
int f(int arg1, int arg2) {
    char x[4];
    ...
}
```

- **Stack frame for function shown at right**
- **Suppose inside f, we write past the end of array x?**
- **What gets overwritten? What happens when we execute ret?**



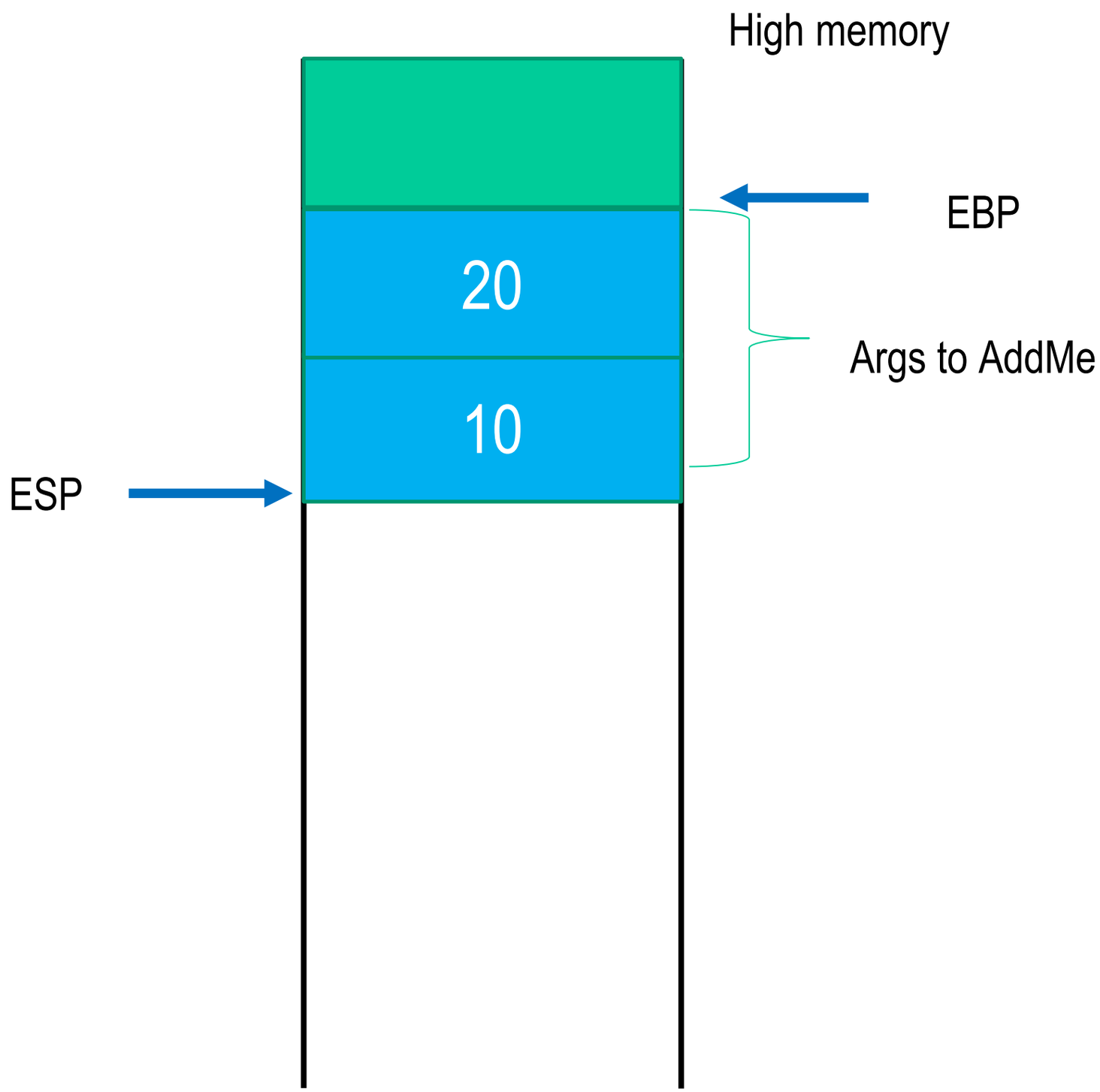
Example:




```
int AddMe(int a, int b)
{
    int c;
    c = a + b;
    return c;
}

main()
{
    AddMe(10,20);
    print…
}
```

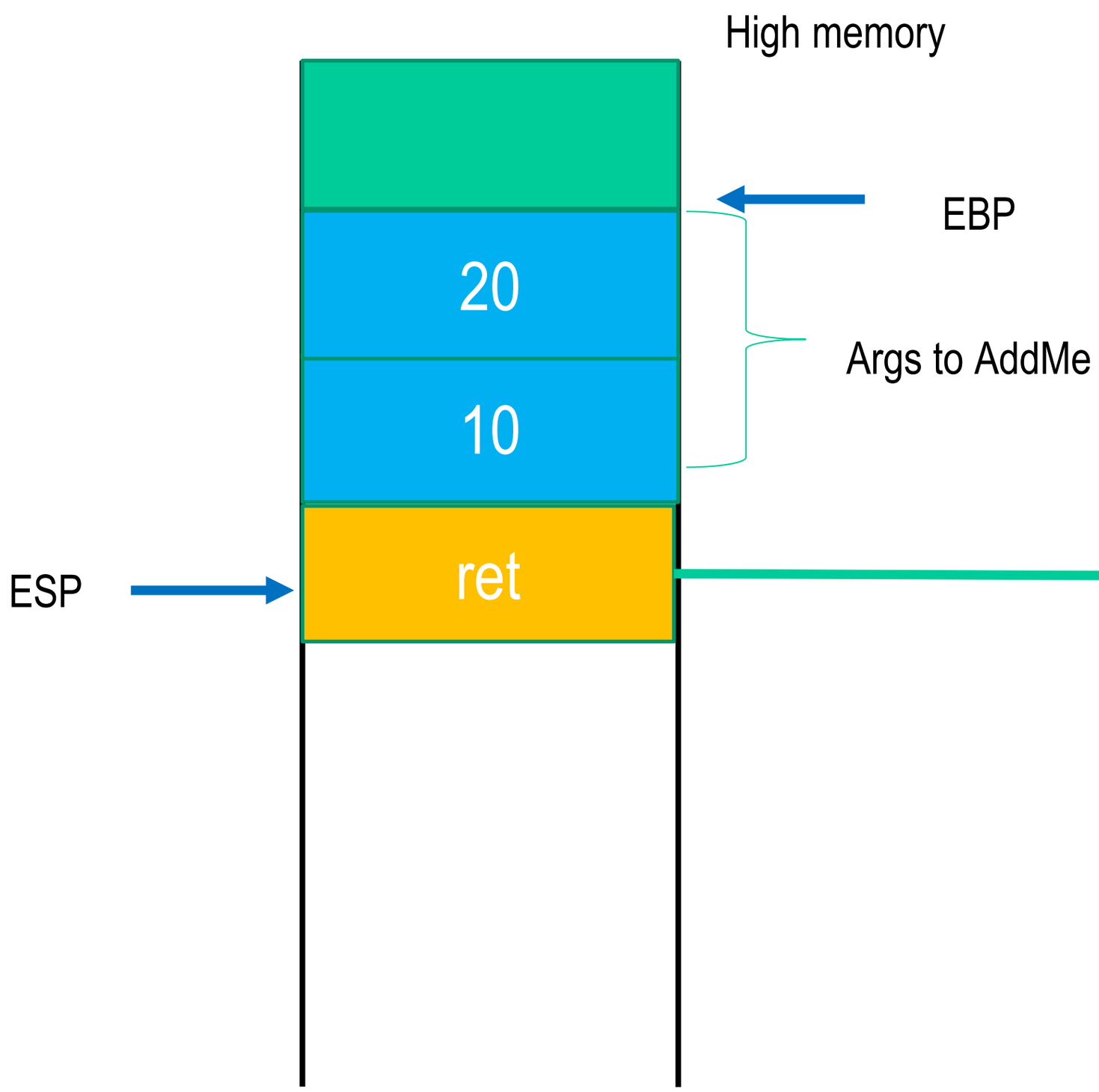
Walking the Call Stack




```
int AddMe(int a, int b)
{
    int c;
    c = a + b;
    return c;
}

main()
{
    AddMe(10,20);
    print…
}
```

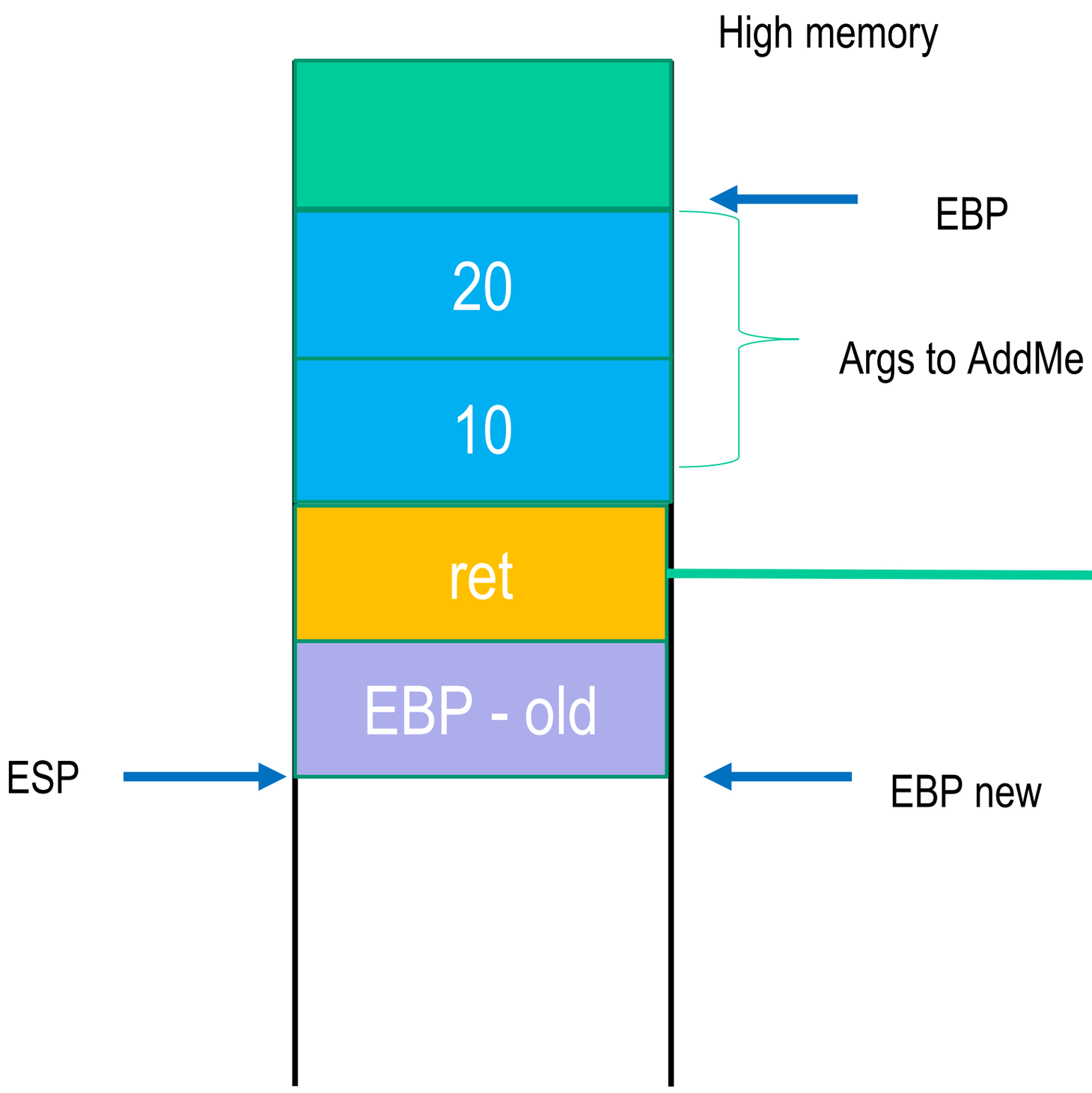
Walking the Call Stack




```
int AddMe(int a, int b)
{
    int c;
    c = a + b;
    return c;
}

main()
{
    AddMe(10,20);
    print…
}
```

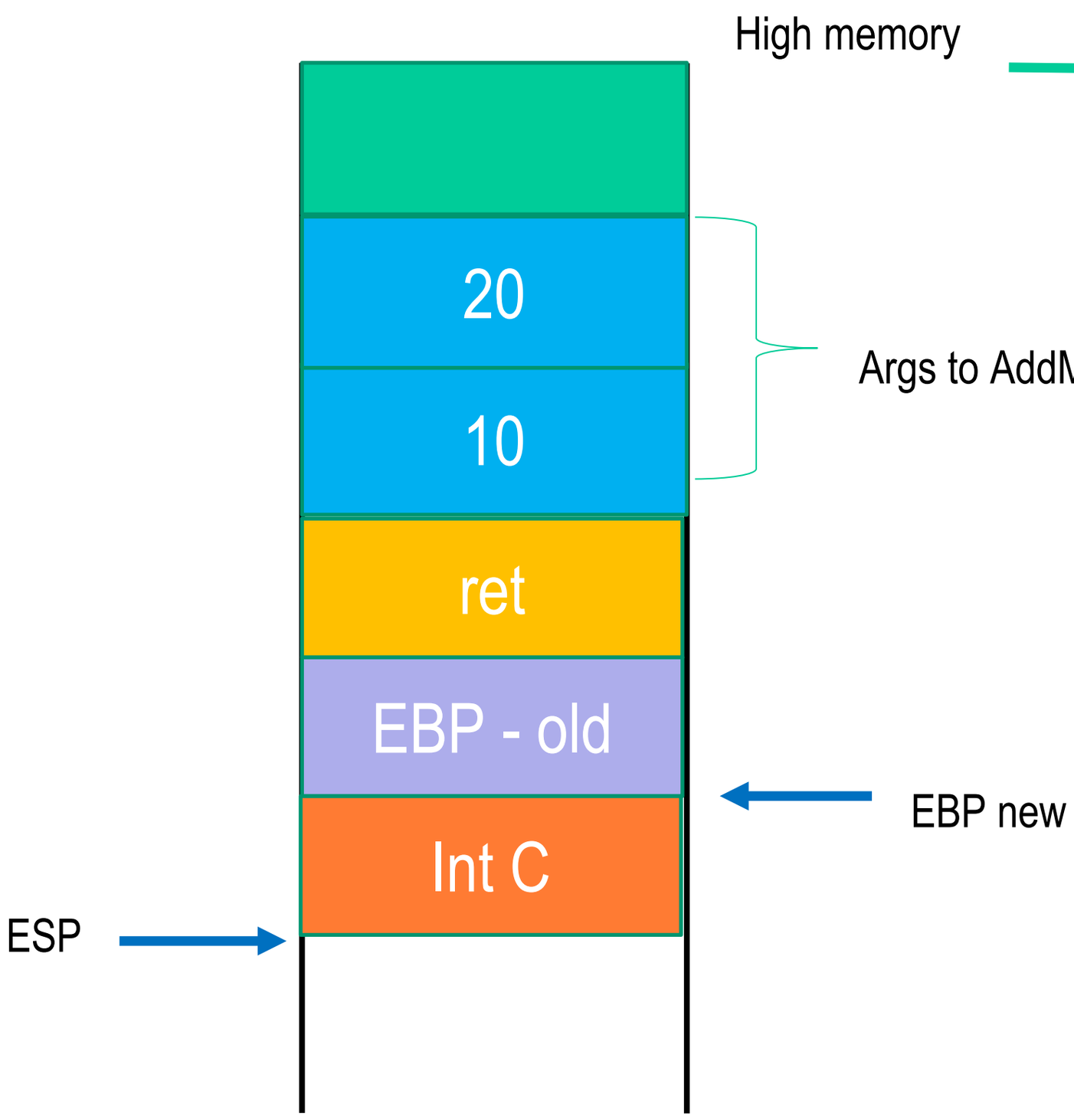
Walking the Call Stack




```
int AddMe(int a, int b)
{
    int c;
    c = a + b;
    return c;
}

main()
{
    AddMe(10,20);
    print…
}
```

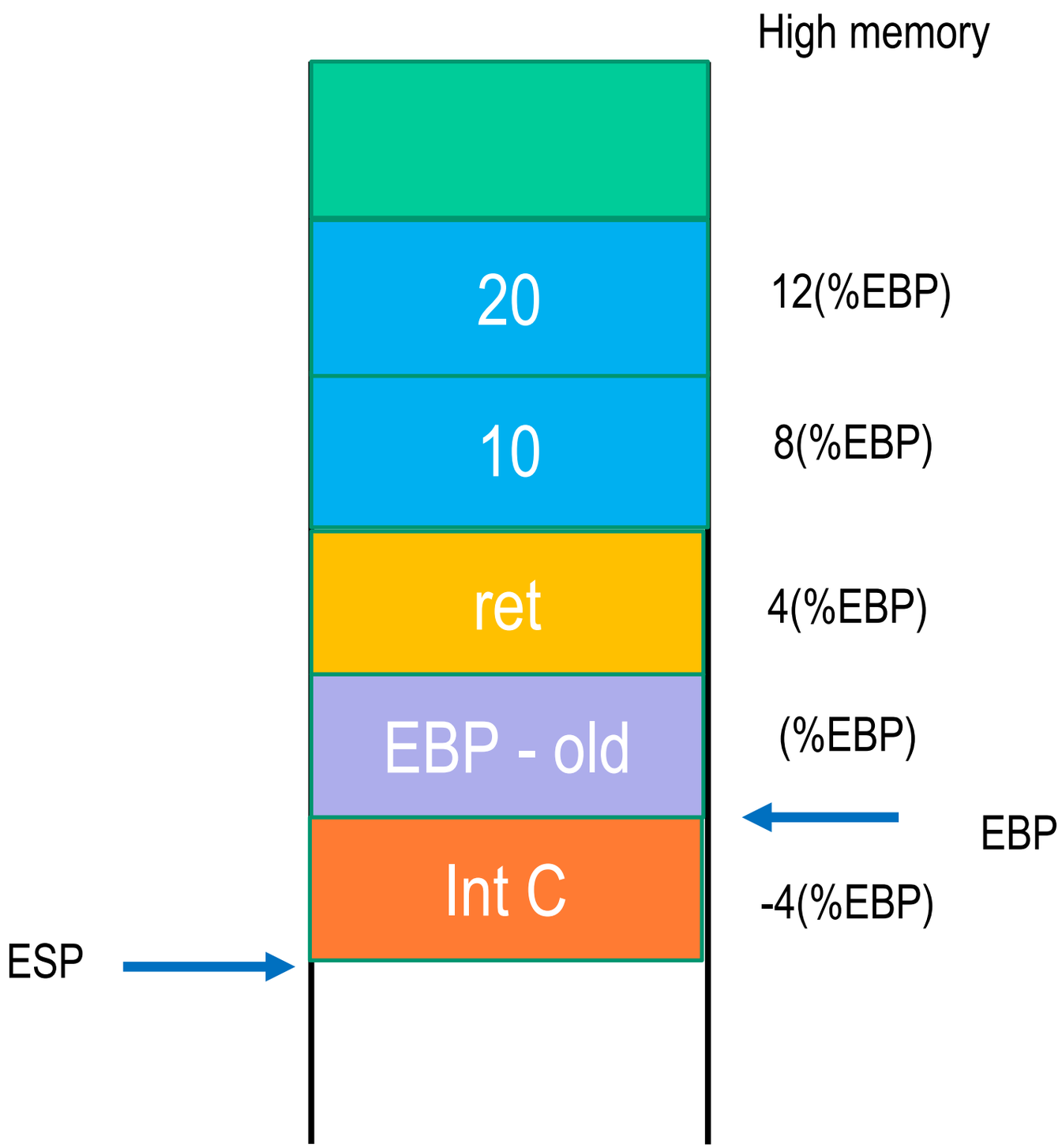
Walking the Call Stack




```
int AddMe(int a, int b)
{
    int c;
    c = a + b;
    return c;
}

main()
{
    AddMe(10,20);
    print…
}
```

To reference data on the stack



Walkthrough

- A great way to understand what's going on is to step through a program in gdb
- Or the VSCode debugger
- You'll also use a debugger to debug the OS, so it's good to become familiar with it
- Another option (at least in assembly):
- http://emu86.pythonanywhere.com

Today: Machine Programming Basics

- **Assembly**
  - More examples
  - Function calls

- **Booting the OS**
  - **BIOS**
  - Assembly Bootstrap
  - C Bootstrap

BIOS

- **When an x86 PC boots, it starts executing the BIOS.**
- **BIOS**
  - Small bit of code stored on the motherboard
  - Prepares the hardware
  - Transfers control to the **OS** (Bootsector)

- **BootSector:**
  - First 512 bytes of the boot disk.
  - Contains the boot loader

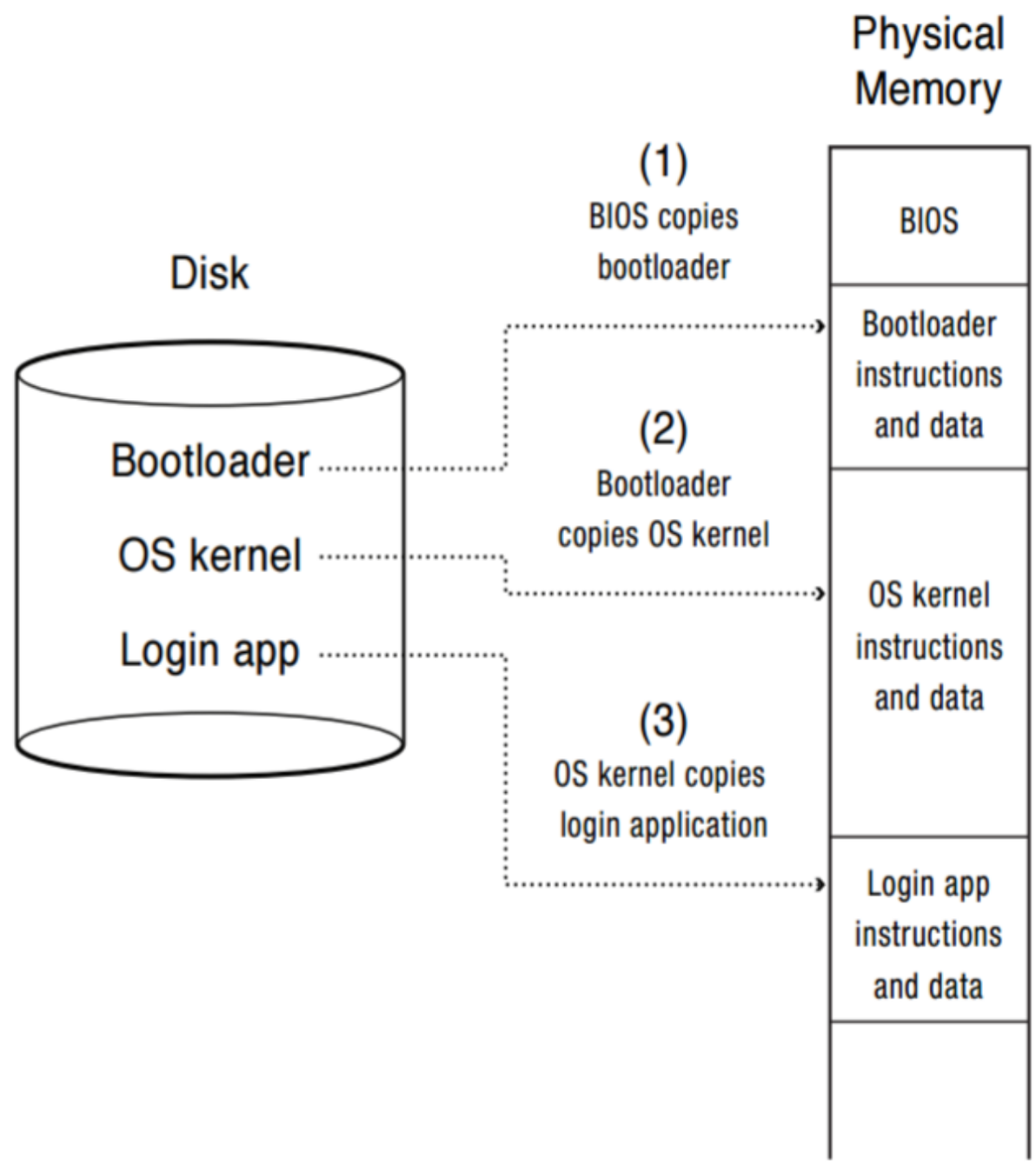
xv6 Bootloader

- Note: x86 processors start out in 16-bit *real mode* (pretends to be an 8088 processor from 1979)
- **BIOS** loads the bootloader (512 bytes) at 0x7c00
- **Bootloader's** job:
  - Put the processor in a more modern operating mode(386 from 8088),
  - Load the xv6 **kernel** from disk into memory,
  - Transfer **control** to the **kernel**.

- The **xv6 boot loader** comprises two parts:
  - **Assembly Bootstrap(bootasm.S:** written in a combination of 16-bit and 32-bit x86 assembly
  - **C Bootstrap(bootmain.c);** written in C

How to bootstrap?

- Power up a computer
- Processor reset
  - Set to known state
  - Jump to ROM code (for x86 PC, this is **BIOS**)
- Load in the boot loader from stable storage
- Jump to the boot loader
- Load the rest of the operating system
- Initialize and run



Today: Machine Programming Basics

- **Assembly**
  - More examples
  - Function calls
- **GDB**
- **Booting the OS**
  - BIOS
  - **Assembly Bootstrap**
  - C Bootstrap

Preliminaries

- **During the BIOS's hardware initialization, it enabled hardware interrupts**
  - We don't know what its interrupt handlers look like, and we don't want to deal with interrupts yet
  - Use **cli** instruction to disable interrupts.
- **We also don't know the state of segment registers, so we clear them**

Preliminaries

```
.code16                         # Assemble for 16-bit mode
.globl start
start:
cli                             # BIOS enabled interrupts; disable

# Zero data segment registers DS, ES, and SS.
xorw    %ax,%ax                 # Set %ax to zero
movw    %ax,%ds                 # -> Data Segment
movw    %ax,%es                 # -> Extra Segment
movw    %ax,%ss                 # -> Stack Segment
```

Clear Interrupts

Clear Segments

Transitioning into Protected Mode

- To run **32-bit code**, we need to transition into ***protected mode***
- A key feature here is that the **meaning of the segment registers** (%cs, %ds, etc.) **changes**
- Instead of just holding extra bits for memory addresses, they *select* a particular *segment descriptor*
  - *Segment descriptor* gives the **base address, size**, and **permissions** of that memory segment

Segments in xv6

- As with most modern OSes, xv6 makes **very little use** of segmentation
- The segments it defines just give a 1-1 mapping between logical and physical addresses
- Later, it will set up the **paging hardware** and use **virtual addresses**

```
# Switch from real to protected mode.
# Use a bootstrap GDT(Global Descriptr
# table)  that makes
# virtual addresses map directly to
# physical addresses so that the
# effective memory map doesn't change
# during the transition.
lgdt    gdtdesc
movl    %cr0, %eax
orl     $CR0_PE, %eax
movl    %eax, %cr0
```

Protected Mode

- **Finally we can jump into protected mode. The change happens only once a segment register is modified, so we execute a *long jump* to set %cs**

- **From here on out, we're executing 32-bit x86 code**

```
ljmp    $(SEG_KCODE<<3), $start32
```

Onward to C

- Finally, we can set up the stack and jump into C code
- The stack pointer is set to 0x7c00 – the address of the start of the bootloader

```
# Set up the stack pointer and call into C.
movl    $start, %esp
call    bootmain
```

Note:

- **The boot loader has “nursed” the processor**
  - Through an evolution from **8088 (16 bit addresses)**
  - Through **80286 (20 bit addresses)**
  - To **80386 (32 bit addresses)**

bootasm.S

```asm
#include "asm.h"
#include "memlayout.h"
#include "mmu.h"

# Start the first CPU: switch to 32-bit protected mode, jump into C.
# The BIOS loads this code from the first sector of the hard disk into
# memory at physical address 0x7c00 and starts executing in real mode
# with %cs=0 %ip=7c00.

.code16                     # Assemble for 16-bit mode
.globl start
start:
  cli                       # BIOS enabled interrupts; disal

  # Zero data segment registers DS, ES, and SS.
  xorw    %ax,%ax           # Set %ax to zero
  movw    %ax,%ds           # -> Data Segment
  movw    %ax,%es           # -> Extra Segment
  movw    %ax,%ss           # -> Stack Segment

  # Physical address line A20 is tied to zero so that the fir
  # with 2 MB would run software that assumed 1 MB.  Undo tha
seta20.1:
  inb     $0x64,%al               # Wait for not busy
  testb   $0x2,%al
  jnz     seta20.1

  movb    $0xd1,%al               # 0xd1 -> port 0x64
  outb    %al,$0x64

seta20.2:
  inb     $0x64,%al               # Wait for not busy
  testb   $0x2,%al
  jnz     seta20.2

  movb    $0xdf,%al               # 0xdf -> port 0x60
  outb    %al,$0x60

  # Switch from real to protected mode.  Use a bootstrap GDT that makes
  # virtual addresses map directly to physical addresses so that the
  # effective memory map doesn't change during the transition.
  lgdt    gdtdesc
  movl    %cr0, %eax
  orl     $CR0_PE, %eax
  movl    %eax, %cr0

  # Complete transition to 32-bit protected mode by using long jmp
  # to reload %cs and %eip.  The segment descriptors are set up with no
  # translation, so that the mapping is still the identity mapping.
  ljmp    $(SEG_KCODE<<3), $start32

.code32  # Tell assembler to generate 32-bit code now.
start32:
  # Set up the protected-mode data segment registers
  movw    $(SEG_KDATA<<3), %ax    # Our data segment selector
  movw    %ax, %ds                # -> DS: Data Segment
  movw    %ax, %es                # -> ES: Extra Segment
  movw    %ax, %ss                # -> SS: Stack Segment
  movw    $0, %ax                 # Zero segments not ready for use
  movw    %ax, %fs                # -> FS
  movw    %ax, %gs                # -> GS

  # Set up the stack pointer and call into C.
  movl    $start, %esp
  call    bootmain
```

Today: Machine Programming Basics

- **Assembly**
  - More examples
  - Function calls
- **GDB**
- **Booting the OS**
  - BIOS
  - Assembly Bootstrap
  - **C Bootstrap**

bootmain.c

- Reads in the kernel header from the hard disk
- Parses the kernel headers – it's a standard ELF (Executable and Linkable Format) file that gcc emits
- Reads each section (kernel code, data) into memory at the address specified in the ELF headers
- Finally, calls the *entry point* defined in the ELF file

- **Bootmain.c**

```c
void
bootmain(void)
{
  struct elfhdr *elf;
  struct proghdr *ph, *eph;
  void (*entry)(void);
  uchar* pa;

  elf = (struct elfhdr*)0x10000;  // scratch space

  // Read 1st page off disk
  readseg((uchar*)elf, 4096, 0);

  // Is this an ELF executable?
  if(elf->magic != ELF_MAGIC)
    return;  // let bootasm.S handle error

  // Load each program segment (ignores ph flags).
  ph = (struct proghdr*)((uchar*)elf + elf->phoff);
  eph = ph + elf->phnum;
  for(; ph < eph; ph++){
    pa = (uchar*)ph->paddr;
    readseg(pa, ph->filesz, ph->off);
    if(ph->memsz > ph->filesz)
      stosb(pa + ph->filesz, 0, ph->memsz - ph->filesz);
  }

  // Call the entry point from the ELF header.
  // Does not return!
  entry = (void(*)(void))(elf->entry);
  entry();
}
```

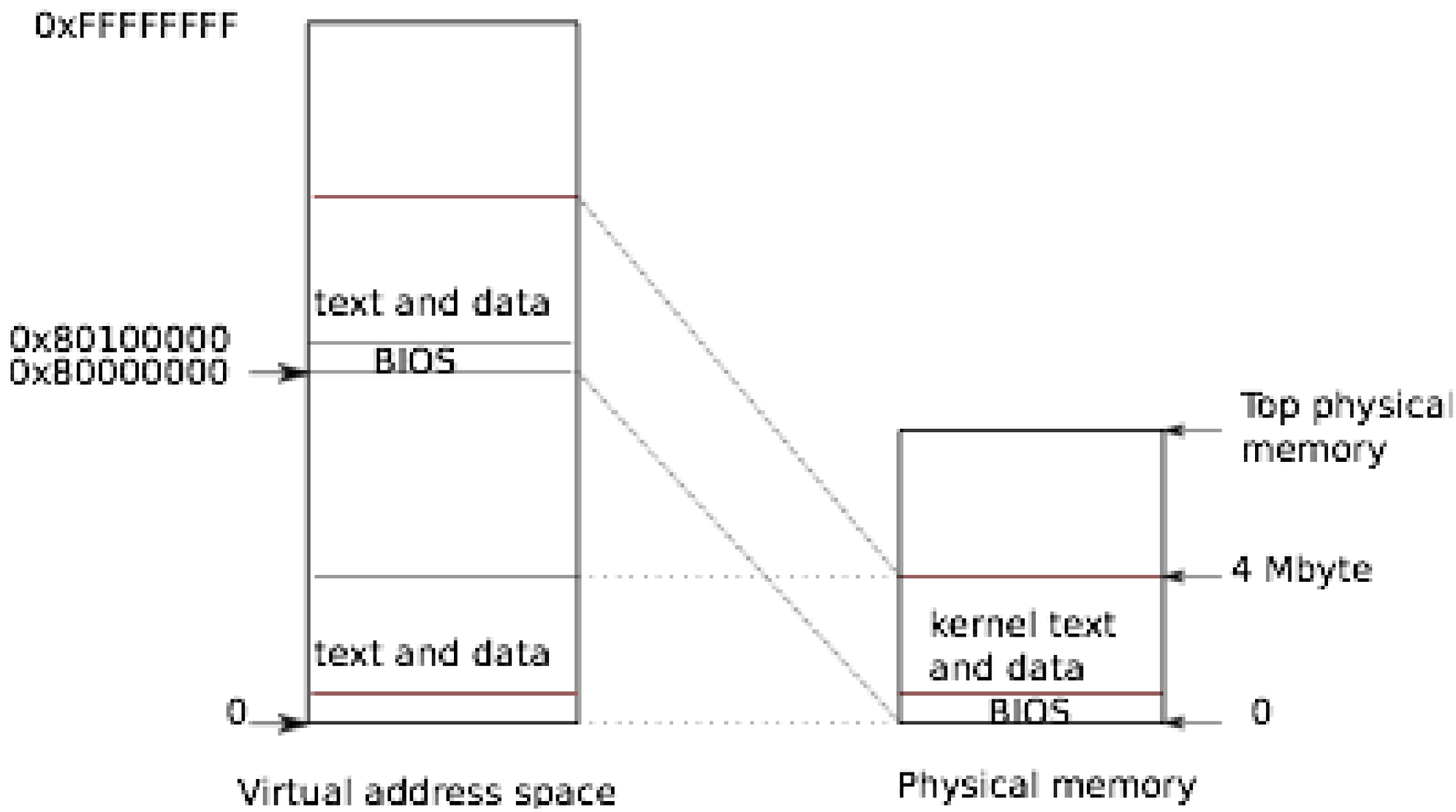
kernel.ld

- The kernel will eventually be loaded at 0x80100000 (virtual)
- For now though, we have only physical memory, not virtual memory
- Luckily, we can use a *linker script* to specify the physical load address, along with the entry point

Kernel Entry Point

- Defined in entry.S
- This is where we finally turn on paging, but for now we just use a very simple mapping:

  0x80000000 => 0x00000000


0xFFFFFFFF
text and data
0x80100000
0x80000000
BIOS
text and data
0
Virtual address space
Top physical memory
4 Mbyte
kernel text and data
BIOS
0
Physical memory

Kernel Main

```
// Bootstrap processor starts running C code here.
// Allocate a real stack and switch to it, first
// doing some setup required for memory allocator to work.
int
main(void)
{
  kinit1(end, P2V(4*1024*1024)); // phys page allocator
  kvmalloc();      // kernel page table
  mpinit();        // collect info about this machine
  lapicinit();
  seginit();       // set up segments
  cprintf("\ncpu%d: starting xv6\n\n", cpu->id);
  picinit();       // interrupt controller
  ioapicinit();    // another interrupt controller
  consoleinit();   // I/O devices & their interrupts
  uartinit();      // serial port
  pinit();         // process table
  tvinit();        // trap vectors
  binit();         // buffer cache
  fileinit();      // file table
  ideinit();       // disk
  if(!ismp)
    timerinit();   // uniprocessor timer
  startothers();   // start other processors
  kinit2(P2V(4*1024*1024), P2V(PHYSTOP)); // must come after startothers()
  userinit();      // first user process
  // Finish setting up this processor in mpmain.
  mpmain();
}
```

Creating the First Process

- **This happens in userinit() – after we've initialized lots of other hardware [proc.c:79]**
- **The first process will run the code in initcode.S**
- **To do that it needs:**
  - A process structure filled out and assigned a slot in the process table (allocproc())
  - An address space (setupkvm())

initcode.S

```
# exec(init, argv)
.globl start
start:
  pushl $argv
  pushl $init
  pushl $0  // where caller pc would be
  movl $SYS_exec, %eax
  int $T_SYSCALL
[...]
# char init[] = "/init\0";
init:
  .string "/init\0"
```